GIST: Gradient-Identical Subgraph Training for Streaming Graph Neural Networks

Sahil Narkhede, Prajjwal Nijhara, Dip Sankar Banerjee
Department of Computer Science and Engineering, Indian Institute of Technology Jodhpur, India
Email: {b23cs1060, d23cse005, dipsankarb}@iitj.ac.in

***Abstract*—Graph neural networks (GNNs) increasingly serve predictions over evolving graphs, and as the data drifts, a deployed model must be repaired while it keeps serving. Incremental inference systems keep embeddings fresh but cannot correct the model itself, streaming retraining systems correct it at the cost of passes over the whole graph, and scalable training methods cut this cost by sampling, partitioning, or reusing stale activations. However, the real obstacle is that a gradient step on a GNN materializes the multi-hop neighborhood of its training nodes, which covers most of the graph even when only a small region has drifted. Existing methods reduce this cost only by approximating the gradient. We present GIST, a runtime that repairs an unmodified, deployed GNN at a cost that follows the retrained nodes rather than the whole graph. GIST exploits the locality of message passing to compute each gradient step only on the subgraph that the retrained nodes depend on, which yields a gradient identical to the gradient on the graph from which this subgraph is extracted. We evaluate GIST on 20 real-world graphs for node classification and link prediction, with four widely used GNN architectures. Compared with state-of-the-art sampled retraining, GIST materializes up to 49% fewer nodes, uses up to 43% less memory, and repairs up to 1.77× faster, while achieving comparable accuracy.**

I. Introduction

Graph neural networks (GNNs) have become the standard approach for learning over relational data [1]–[4] and are deployed inside data systems that operate on evolving graphs, such as user–item graphs for recommendation [5], transaction graphs for fraud detection [6], and citation graphs [7]. In such deployments, the graph arrives as a stream of edge, node, and attribute updates [8], while the model is trained once on a snapshot and then served. Two families of systems support these deployments. Incremental inference systems refresh predictions by recomputing only the embeddings that an update affects [9]–[11], whereas retraining systems update the model itself, either once sufficient change has accumulated [12], [13] or on every new snapshot [14]. The former never corrects the model, and the latter corrects it at the cost of passes over the graph.

Correcting the model becomes necessary because the data a deployed model observes drifts away from the data it was trained on [15], [16]. On graphs, node features shift, edges form between previously unrelated communities, and the mapping from a node's neighborhood to its label changes [17], [18]. In the last case, refreshing embeddings does not help, since the error lies in the model parameters, and only gradient updates can repair the model. However, a gradient step on an L-layer GNN processes every node within L hops of its training nodes, which for nodes spread across the graph approaches the entire graph [19], [20], and under drift this cost recurs on the same hardware that serves predictions. In this paper, we study how to repair the parameters of a deployed GNN under drift while it continues to serve.

Such a repair must meet three objectives together, namely ❶ **parameter adaptation**, so that the model itself is corrected, ❷ **bounded update cost**, so that the cost of a repair follows the retrained nodes rather than the size of the graph, and ❸ **identical updates**, so that a repair computed on a smaller part of the graph yields exactly the gradient of the graph it is taken from. Existing approaches meet these objectives only in part. Incremental inference systems such as InkStream [9] and Ripple [10] track the affected embeddings and produce the same embeddings as a recomputation from scratch, meeting ❷ and ❸, but they keep the parameters fixed and cannot respond when only the labels drift. Streaming retraining [12]–[14] and continual graph learning [21]–[23] meet ❶ and decide what to train on, but each gradient step still spans the full neighborhood of its training nodes regardless of the size of the drifted region, violating ❷. Continual learning also assumes task boundaries that a stream does not provide [24], [25]. Scalable training methods such as Cluster-GCN [19], GraphSAINT [26], and GNNAutoScale [27] meet ❷ by dropping edges, sampling neighborhoods, or reusing stale activations, which suits training from scratch but violates ❸ when repairing a trained model, since the approximate gradient moves predictions away from the intended ones. To the best of our knowledge, no existing approach adapts the parameters of a deployed GNN at a cost bounded by the retrained nodes without approximating the gradient.

A. Motivation, Challenges and Contributions

To examine how far existing approaches are from meeting all three objectives, we measure what one repair costs and whether it computes the intended update, on `ogbn-arxiv` ordered by publication year and with label drift injected into `Cora`, `PubMed`, and `ogbn-arxiv`. When the labels of a community change, a model with fixed parameters classifies only 0.5–7% of the affected holdout nodes correctly, whereas twenty gradient steps restore 32–62%. A parameter update is therefore necessary, but its cost does not follow the drifted region. For a repair on $8{,}192$ training nodes, whose 2-hop neighborhood covers 84% of `ogbn-arxiv`, sampled retraining inside the stream spends 63% of a GAT repair and

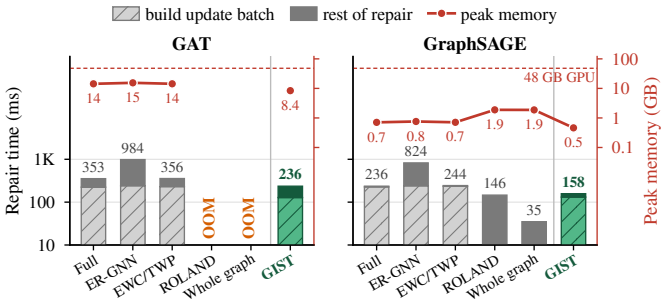


Fig. 1: Time and memory of one repair on ogbn-arxiv.

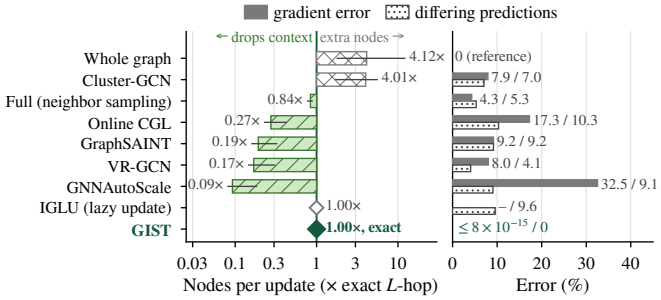


Fig. 2: Cost and exactness of the same repair on ogbn-arxiv.

96% of a GraphSAGE repair building the sampled batch on the host (Fig. 1), and continual learning methods add up to 3.5× on top. Whole-graph retraining, as in InstantGNN [12] and D3-GNN [13], avoids the sampling and is fast while the graph fits, but runs out of GPU memory for GAT, as does ROLAND. Since the repair runs on the GPU that serves predictions, it also raises the 99th-percentile inference latency by 2.4–5.3× while it runs.

Methods that bound this cost do so by changing the repair. Fig. 2 applies each of them to the same repair and compares the nodes it materializes with the L-hop neighborhood of the retrained nodes, which is exactly the part of the graph that the gradient depends on. Neighbor sampling, GraphSAINT [26], and historical-embedding methods such as GNNAutoScale [27] materialize only 9–84% of this neighborhood, and each of them computes a different gradient, with a relative error of 4.3–32.5%. Cluster-GCN [19] materializes 4× the neighborhood and is still inexact, since its clusters cut edges inside it. IGLU [28] materializes exactly this neighborhood but reuses activations from earlier steps, so the right subgraph alone does not make an update exact. After twenty steps, the resulting models disagree with the exact repair on 4–11% of the holdout predictions, and the only exact update, on the whole graph, materializes 4.1× the nodes that the gradient needs.

Existing approaches therefore fall short in two ways. Approaches that keep the parameters fixed cannot repair a model whose learned mapping has become wrong, and approaches that update the parameters either pay for more of the graph than the update needs or bound this cost by computing a different update. Both overlook that the output of an L-layer GNN at a node depends only on its L-hop neighborhood, so the computation that a repair requires is determined by the retrained nodes alone. No approach in Fig. 2 computes its update on exactly this neighborhood, whose size in turn depends on which nodes are retrained. The key challenge is therefore to confine each repair to the part of the graph on which the retrained nodes depend, while keeping the update exact, which leads to the following questions.

Q1 Which part of the graph does a gradient step on the retrained nodes depend on, and how can the runtime compute the update on that part alone?

Q2 Under which conditions is such a localized update identical to the update on the full graph, and how can it remain exact for normalized layers and temporal GNNs?

Q3 How can the runtime decide when to repair and which nodes to retrain from the stream itself, without modifying the deployed model?

We address these questions with GIST[1] (Gradient-Identical Subgraph Training), a runtime that sits between the graph stream and an unmodified message-passing GNN, detects drift from the outputs of the model's own forward pass, and computes each repair on the subgraph induced by the L-hop neighborhood of the retrained nodes. This paper makes the following contributions.

1) We analyze the cost of a parameter update on a deployed GNN. We find that this cost is set by the L-hop expansion of the retrained nodes rather than by their number, and that nodes from a coherent region share most of it.
2) We design an exact localized update on the induced L-hop subgraph of the retrained nodes, whose gradient on the retrained nodes is the same as on the parent graph, and keep it exact for normalized layers and snapshot-based temporal GNNs by taking edge coefficients from the parent graph and freezing normalization statistics.
3) We introduce a repair trigger that monitors the outputs of the served model, identifies the regions that have drifted, and draws the retrained nodes from them together with a rehearsal share from the rest of the graph, while accessing the model only through its forward function, its depth, and a per-layer cost function.
4) We identify five evaluation practices, such as an under-trained starting model, that inflate the reported benefit of retraining, and design an evaluation protocol that controls for each of them.

We evaluate GIST against sampled retraining and thirteen prior update rules on six node-classification streams and fifteen link-prediction graphs with four GNN backbones. The localized gradient matches that of the graph it is extracted from to within 10^{-13} in every audited repair. Compared with state-of-the-art sampled retraining at the same budget, GIST materializes up to 49% fewer nodes, uses up to 43% less memory, and repairs up to 1.77× faster, with statistically equivalent accuracy in 22 of 23 settings. GIST completes repairs on which sampled retraining runs out of GPU memory.

[1]Our source code is available at https://github.com/SPADE-IITJ/GIST

II. Background

A. Existing Works

Approaches for keeping a deployed GNN accurate on an evolving graph differ in whether they update the parameters, whether their cost follows the retrained nodes rather than the graph, whether the result remains exact, and whether they run without task boundaries (Table I). Incremental inference systems [9]–[11] bound their cost by the affected nodes and match full inference, Ripple for linear aggregation and StreamTGN in its exact mode, but keep the parameters fixed. Streaming retraining systems [12]–[14] and dynamic GNNs [29]–[35] update the parameters on the graph, snapshot, or window they select, so their cost follows the graph rather than the change. Continual graph learning [21]–[23] assumes known task boundaries [24], [25] and trains on the full neighborhoods of its training nodes. Since these retraining and continual learning methods do not reduce the work of an update, exactness does not apply to them.

TABLE I: Comparison of approaches to keeping a deployed GNN accurate on an evolving graph.

Framework	Param. update	Bounded cost	Exact result	Task-free	Mechanism
Incremental GNN inference					
InkStream [9]	✗	✓	✓	✓	Incremental update
Ripple [10]	✗	✓	✓	✓	Delta propagation
StreamTGN [11]	✗	✓	✓	✓	Dirty-flag propagation
Streaming retraining and dynamic GNNs					
InstantGNN [12]	✓	✗	✗	✓	Triggered retraining
D3-GNN [13]	✓	✗	–	✓	Dataflow training
ROLAND [14]	✓	✗	–	✓	Incremental training
Continual graph learning					
ER-GNN [21]	✓	✗	–	✗	Experience replay
TWP [22]/EWC [36]	✓	✗	–	✗	Regularization
CaT [23]	✓	✗	–	✗	Condensed replay
Online CGL [20]	✓	✓	✗	✓	Neighbor sampling
Scalable GNN training					
Cluster-GCN [19]	✓	✓	✗	✗	Graph partitions
GraphSAINT [26]	✓	✓	✗	✗	Subgraph sampling
VR-GCN [37]/GAS [27]	✓	✓	✗	✗	Cached activations
IGLU [28]	✓	✓	✗	✗	Lazy updates
GIST (ours)	✓	✓	✓	✓	**Induced subgraph**

Scalable training [19], [26]–[28], [37] and online continual graph learning [20] bound the cost by dropping edges, sampling neighborhoods, or reusing stale activations, and therefore compute a different gradient. SAGN [38], [39] adapts the model per subgraph, but over correlated data streams rather than a single evolving graph. To the best of our knowledge, no existing approach satisfies all four properties together.

B. Preliminaries

1) Graph streams and drift: A graph $G_t = (V_t, E_t, X_t)$ with node features x_v and labels y_v evolves through a stream of events Δ_t, namely insertions and deletions of nodes and edges and changes to features or labels. A GNN f_θ is trained once on G_0 over training nodes V^{tr} and then serves the stream, while disjoint holdout nodes V^{ho} only measure accuracy. Drift appears in three ways [15], [17], [18]. Feature drift and structural drift change the inputs of the model, so refreshing the affected embeddings suffices while the learned mapping from a neighborhood to a label still holds. Label drift changes this mapping itself, so the error lies in θ, and only a parameter update can repair it. GIST performs this repair and leaves the refresh of embeddings to incremental inference.

2) Message-passing GNNs: An L-layer message-passing GNN computes the representation of each node v from those of its neighbors,

$$h_v^{(\ell)} = \phi^{(\ell)}\Big(h_v^{(\ell-1)},\ \bigoplus\nolimits_{u:(u,v)\in E} \psi^{(\ell)}\big(h_v^{(\ell-1)}, h_u^{(\ell-1)}, w_{uv}\big)\Big), \quad (1)$$

with $h_v^{(0)} = x_v$, a permutation-invariant aggregator $\bigoplus$, and edge coefficients w_{uv}. GCN [1], GraphSAGE [2], GAT [3], and GIN [4] are instances of this form. Each layer moves information by one hop, so the output $f_\theta(v) = h_v^{(L)}$ depends only on the nodes within L hops of v. For a set of nodes S of a graph P, we write $\mathcal{N}_L(S; P)$ for its L-hop neighborhood, that is, the nodes of P with a path of at most L edges into S.

3) Parameter updates and their cost: A repair trains on a set of retrained nodes $S \subseteq V^{\mathrm{tr}}$, which we call the *roots*, with the loss placed on the roots only,

$$\mathcal{L}(\theta; S, P) = \tfrac{1}{|S|} \textstyle\sum_{v\in S} \ell\big(f_\theta(v; P), y_v\big), \quad (2)$$

where the *parent graph* P is the current graph G_t or a batch sampled from it. The gradient of (2) needs the input features of every node of $\mathcal{N}_L(S; P)$ and, at layer ℓ, the representations of the nodes within $L - \ell$ hops of the roots. The cost of an update is therefore set by the neighborhood of its roots and not by their number. Roots from one coherent region share most of their neighbors and keep this neighborhood small, whereas roots spread across the graph share few, which is why $8{,}192$ uniform roots covered 84% of `ogbn-arxiv` in Section I-A.

4) Gradient-identical updates and problem statement: Locality also determines which part of P an update needs. Let $H = P[\mathcal{N}_L(S; P)]$ be the subgraph induced by the L-hop neighborhood of the roots, namely these nodes and every edge of P between them. H holds everything on which the output at a root depends, so the outputs at the roots, their loss, and its gradient are the same on H as on P for every θ,

$$\nabla_\theta \mathcal{L}(\theta; S, H) \;=\; \nabla_\theta \mathcal{L}(\theta; S, P). \quad (3)$$

We call such an update *gradient-identical*, since it is the update on P itself, computed on a smaller graph. The equality needs three conditions, namely that H spans the full depth L, that the loss is placed on the roots only, since the nodes at the border of H miss part of their neighborhoods, and that every layer reads the graph only through the neighbors of a node, with w_{uv} given as an input rather than recomputed from H. The equality is also relative to P, so the update on H is the whole-graph update when $P = G_t$ and the batch update when P is a sampled batch.

We therefore seek a runtime that repairs a deployed model, accessible only through its forward function, depth L, and per-layer cost, while it serves, such that ❶ every repair updates θ, ❷ its cost follows H rather than the graph, and ❸ its update is gradient-identical to the update on P.

III. Our Approach

A gradient step on a set of roots depends only on the subgraph H induced by their L-hop neighborhood, and under three conditions the step on H is identical to the step on the parent graph P (Section II-B4). GIST builds on this property a runtime that repairs a deployed GNN while it continues to serve. As shown in Fig. 3, GIST sits between the graph stream and an unmodified GNN and runs two paths. The serving path executes on every step and consists of the model's own forward pass. The repair path executes only when drift is detected. It selects the nodes to retrain, computes a gradient step on their induced subgraph, and writes the updated parameters θ back to the model. Each repair therefore updates θ, which meets ❶, performs work bounded by the neighborhood of its roots rather than by the size of the graph, which meets ❷, and computes the same gradient as on P, which meets ❸. The remainder of this section describes the model interface and the runtime loop, the selection of the roots of a repair, the localized update, the cost model, and the configuration of GIST.

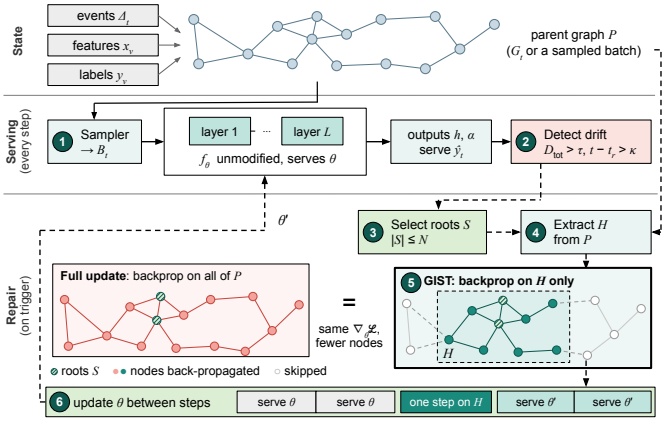


Fig. 3: Architecture of GIST.

A. System Overview

1) Model Interface: GIST treats the deployed model as a black box with a small interface. The architecture and the training code of the model remain unchanged, and the runtime requires only three things from the model, namely its forward function $f_\theta(X, E, w)$, its receptive-field depth L, and a per-layer cost function $c_\ell(n, m)$ that it uses for accounting. Two further inputs are optional. Layers that normalize over the graph accept precomputed edge coefficients w, which keep the update exact (Section III-C), and models with attention expose their attention coefficients α, which serve as an additional drift signal (Section III-B1). GAT, GraphSAGE, GCN, and GIN run through this interface, and so do the snapshot-based temporal models EvolveGCN-O [29], T-GCN [31], and GConvGRU [30]. T-GCN and GConvGRU carry their recurrent node state as a node input, whereas EvolveGCN-O evolves its weights across snapshots and keeps no node state. Since GIST never inspects the internals of the model, all of these models run through a single code path.

Algorithm 1 One step of the GIST runtime

Require: model f_θ with interface (L, c_ℓ, w); budget N; rehearsal share ρ; threshold τ; cooldown κ; last repair t_r

1: $G_t \leftarrow \textsc{Apply}(G_{t-1}, \Delta_t)$
2: $B_t \leftarrow \textsc{Sample}(G_t, R_t)$ ▷ L-hop, fan-out limited
3: $(h, \hat{y}, \alpha) \leftarrow f_\theta(B_t)$ ▷ no gradient
4: **serve** $\hat{y}_v$ for $v \in R_t$
5: $D_{\text{tot}} \leftarrow \textsc{Detect}(h, \hat{y}, \alpha)$ ▷ Eq. (4)
6: **if** $D_{\text{tot}} > \tau$ **and** $t - t_r > \kappa$ **then**
7: $\quad \mathcal{C}_t \leftarrow \textsc{Regions}(h)$ ▷ Eq. (5)
8: $\quad S \leftarrow \textsc{Draw}\big((1-\rho)N, \bigcup \mathcal{C}_t \cap V^{\text{tr}}\big) \cup \textsc{Draw}\big(\rho N, V^{\text{tr}}\big)$
9: $\quad H \leftarrow P\big[\mathcal{N}_L(S; P)\big]$, with w sliced from P
10: $\quad \theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta; S, H)$ ▷ one Adam step, norm statistics frozen
11: $\quad t_r \leftarrow t$
12: **end if**

2) Runtime Loop: Each step of the runtime follows Algorithm 1. On the serving path (lines 1–4), the events Δ_t update the graph, the sampler draws the L-hop batch B_t of the nodes R_t that the step serves, and the model computes its predictions. The detector (line 5) reads only the outputs of this forward pass, so monitoring adds no forward pass of its own. The repair path (lines 6–11) executes only when the detector fires and a cooldown has passed since the last repair. A repair runs on the same thread as serving, between two consecutive steps, so every prediction is served either by the old parameters or by the updated ones, never by a mix of the two. Only the refresh of the graph partition used for root selection runs in the background, off the serving path.

B. Drift-Triggered Root Selection

Before it computes an update, a repair makes two decisions, namely when to fire and which nodes to retrain. GIST makes both decisions from the outputs of the serving path, without labels and without an additional forward pass.

1) Drift Detection: Labels in a stream arrive late or not at all, so GIST detects drift from the outputs of the forward pass that serving has already computed. The detector tracks three signals, each against an exponentially weighted reference with rate γ. The first is the change in the mean norm of the representations, $D_{\text{emb}} = \left|\frac{1}{|B_t|}\sum_v \|h_v\| - \bar{n}\right|$. The second is the change in the predicted class distribution, $D_{\text{pred}} = \|p_t - \bar{p}\|_2$. The third, for models with attention, is the change in the mean attention coefficient, D_{att}. These signals differ in scale by orders of magnitude, and a weighted sum of their raw values would be dominated by the largest one regardless of the weights. GIST therefore standardizes each signal online with its running mean and variance, written $z(\cdot)$, before combining them as

$$D_{\text{tot}}(t) = \frac{a\, z(D_{\text{emb}}) + b\, z(D_{\text{pred}}) + c\, z(D_{\text{att}})}{\|(a, b, c)\|_2}. \tag{4}$$

For roughly independent signals, the denominator keeps D_{tot} at unit variance for any choice of weights. The threshold τ therefore reads in standard deviations and keeps its meaning when a signal is absent, as for models without attention, where $c = 0$. GIST sets τ for each model and graph to the 95th percentile of D_{tot} on calibration runs without any repair, so that the detector flags 5% of the steps of a drift-free stream.

A repair fires when $D_{\text{tot}} > \tau$ and more than κ steps have passed since the last repair. Only a repair that takes a gradient step restarts this cooldown, so a repair that finds no nodes to retrain does not suppress the next one. The detector decides only whether to repair at a given step, so a timer, an external alert, or the arrival of new labels on training nodes can replace it without any change to the rest of the runtime or to the update that follows. Such a replacement is required under drift that changes labels while leaving the inputs unchanged, since the outputs of the model then do not change.

2) Region Selection: Once a repair fires, GIST decides which nodes to retrain. The cost of an update is set by the L-hop neighborhood of its roots, and roots from one coherent region share most of this neighborhood (Section II-B3). GIST therefore draws its roots from regions of the graph rather than from the whole graph. It partitions V into structural regions $\mathcal{C} = \{C_1, \dots, C_k\}$ with METIS [40]. The partition is recomputed in the background on a snapshot of the graph and installed a fixed number of steps later. A greedy maximum-overlap matching carries the identity of each region across refreshes, so that the drift of a region forms a time series.

To locate the regions that have drifted, each node keeps an exponentially weighted reference $\bar{h}_v$ of its output. When a repair fires, GIST scores each region by the mean displacement of its nodes in the current batch and selects the regions whose score lies in the upper tail,

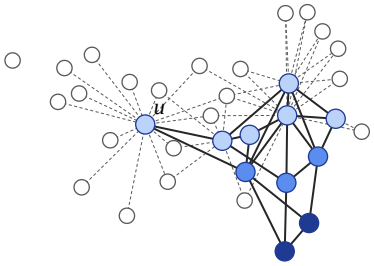
$$q_t(C) = \frac{1}{|C \cap B_t|} \sum_{v \in C \cap B_t} \|h_v - \bar{h}_v\|_2, \quad \mathcal{C}_t = \{C : q_t(C) > \mu_t + k_\sigma \sigma_t\}, \tag{5}$$

where μ_t and σ_t are the mean and the standard deviation of the scores at step t, computed over the regions with at least one node in B_t. Regions without such a node are not scored at this step. The criterion is relative, so it continues to select the tail of the distribution as the absolute scores move. If no region passes it, GIST selects the region with the highest score.

3) Rehearsal: Retraining only inside the selected regions, however, revisits the same nodes while the shared parameters change for every node, so the model forgets the rest of the graph [21]. GIST therefore draws up to $(1-\rho)N$ roots from the selected regions and ρN roots uniformly from V^{tr} as rehearsal (Algorithm 1, line 8). The budget N is an upper bound on the number of roots rather than a quota. When the selected regions hold fewer training nodes than their share, the update has fewer roots, and rehearsal makes up a larger part of it. Rehearsal roots are spread across the graph by construction, so their neighborhoods overlap little and they enlarge the update. The rehearsal share ρ therefore trades the cost of an update against the part of the graph that the update revisits.

C. Localized Update

1) Subgraph Extraction: With the roots S selected, GIST extracts $H = P[\mathcal{N}_L(S; P)]$ on the host, transfers only H to the GPU, and takes the gradient step of (2) on it (Algorithm 1, lines 9–10). Fig. 4(a) illustrates H for two roots and $L = 2$ on a small real graph, where H holds 11 of its 34 nodes. The subgraph contains the roots, their 1-hop and 2-hop neighbors, and every edge of P among these nodes, whereas the edges that lead to nodes outside H are cut. Since H is induced, every node within $L-1$ hops of the roots keeps all of its in-neighbors, together with the edges and edge coefficients that connect them, so its first-layer representation on H equals its representation on P. At the second layer, the same holds for every node within $L-2$ hops, since its in-neighbors now carry exact representations, and after L layers it holds for the roots themselves.



(a) Induced subgraph H in P ($L = 2$)

Max. relative difference between $h^{(\ell)}$ on H and on P

	S	1 hop	2 hops
$h^{(0)}$	0	0	0
$h^{(1)}$	0	0	0.95
$h^{(2)}$	0	0.31	0.91

(b) Measured difference

Fig. 4: Localized update for $L = 2$ on Zachary's karate club. (a) The subgraph H induced by the 2-hop neighborhood of the roots S (dark) in the parent graph P, with 1-hop and 2-hop nodes in lighter shades and nodes outside H in white. Dashed edges are cut from H. (b) Largest relative difference between the representations on H and on P at each layer, measured on a 2-layer GCN in float64.

The set of exact nodes therefore shrinks by one hop at every layer, which the measurements in Fig. 4(b) confirm. Every node of H starts from its exact input features. After the first layer, the roots and their 1-hop neighbors remain exact, whereas the 2-hop nodes lose the contributions of their cut edges and differ from their representations on P by up to 95%. After the second layer, only the roots remain exact. The loss on the roots is therefore the same function of θ on H as on P, and so is its gradient. The update neither reuses stored representations nor drops edges within the receptive field of the roots, so the saving comes only from work that the gradient on the roots never needed.

2) Exactness Conditions: This argument relies on the three conditions of Section II-B4, and GIST enforces each of them at runtime. The first condition is that H spans the full receptive field of the model. GIST takes L from the model interface instead of inferring it, since a subgraph of $L-1$ hops misses an in-neighbor of every node at distance $L-1$ and changes the gradient. For recurrent snapshot models, GIST counts every graph convolution inside a recurrent cell, so GConvGRU, whose cell applies two convolutions in sequence, requires two hops per cell.

The second condition is that the loss is placed on the roots only. As Fig. 4(b) shows, the nodes at the border of H have truncated receptive fields, so GIST computes the loss of (2)

on S alone.

The third condition is that every layer reads the graph only through the in-neighbors of a node. Two kinds of layers read more than that, and GIST corrects both. GCN's symmetric normalization $w_{uv} = (d_u d_v)^{-1/2}$ reads the degree of the sending node, which is too small at the border of H. The border node u in Fig. 4(a), for instance, keeps only 2 of its 16 edges in H, and recomputing the normalization on H instead of taking it from P leaves a relative error of 0.10 in the representations of the roots. Chebyshev convolutions similarly rebuild a scaled Laplacian from the graph they receive. GIST computes these coefficients once on P, slices them with H, and passes them to the layer through the interface with the layer's own normalization disabled. Batch normalization computes its statistics over every node present, so GIST uses its running statistics during a repair. Memory-based temporal GNNs [33], [41], [42] keep a node state that depends on the whole event history, so they have no finite depth L and lie outside the interface.

3) Parent Graph Heuristic: The guarantee holds relative to P, so the choice of P determines both what the update equals and what it costs. With $P = G_t$, the update is the whole-graph gradient of the loss on the roots, at the cost of the full L-hop neighborhood of S. With P a batch sampled from the roots with fixed fan-outs, every node of the batch already lies within L hops of the roots. H then coincides with the batch, the update equals the update on that batch, and the saving comes entirely from the overlap of the roots' neighborhoods. GIST uses the sampled parent during streaming, which bounds the cost of a repair by the fan-outs, and the whole graph when the update must equal the full-graph gradient.

D. Cost Model

1) Repair Cost: The cost of a repair follows from the size of H. Let $n_H = |\mathcal{N}_L(S; P)|$ be its number of nodes and m_H its number of edges. The forward cost $c_\ell(n, m)$ of a layer is linear in both, since a layer applies a dense transform to every node and sends a message along every edge. Charging the backward pass at twice the forward pass, the compute and the activation memory of a repair are

$$F(S) = 3 \sum_{\ell=1}^{L} c_\ell(n_H, m_H), \quad M(S) \propto n_H \sum_{\ell=1}^{L} d_\ell, \tag{6}$$

where d_ℓ is the width of layer ℓ. Both quantities depend on the roots only through their L-hop neighborhood and not through the size of the graph. Roots from a few coherent regions share neighbors, so n_H grows sub-linearly with $|S|$, whereas uniform roots share few neighbors, and n_H approaches the combined size of $|S|$ disjoint neighborhoods. Relative to an update on N uniform roots, a repair of GIST therefore saves through the overlap of its roots' neighborhoods and through any shortfall of $|S|$ below N, and the rehearsal share ρ places its cost between these two extremes.

2) Streaming Cost: Over a whole stream, the serving work adds to the cost of the repairs. Over T steps with E repairs, the runtime takes $T(t_{\text{inf}} + t_{\text{det}}) + \sum_{e=1}^{E} \big(t_{\text{reg}} + t_{\text{upd}}(S_e)\big)$, where t_{inf} is the serving step of the model, t_{det} the detector, t_{reg} the region scoring, and t_{upd} the update, including sampling on the host. The partition refresh runs in the background and does not add to this time. Relative to updates on uniform roots at fixed intervals, which need no detector, the cheaper repairs of GIST pay for its monitoring when

$$E\left(t_{\text{upd}}^{\text{full}} - t_{\text{upd}} - t_{\text{reg}}\right) > T\, t_{\text{det}}, \tag{7}$$

so the end-to-end benefit grows with the saving per repair, $t_{\text{upd}}^{\text{full}} - t_{\text{upd}}$, and with the number of repairs.

E. Configuration and Implementation

The starting model is trained to convergence on the graph that it serves. Initial training runs until the accuracy plateau of each model and graph, on the same symmetric graph that the stream uses, since a model below its plateau would gain accuracy from any repair simply by continuing its training. A repair uses its own learning rate, $\eta = 10^{-3}$, separate from that of the initial training, so that the strength of a repair can be tuned without changing the deployed model. The length of the initial training and the threshold τ are calibrated for each model and graph on runs separate from those used for evaluation. The remaining parameters are fixed across backbones. A repair takes one Adam step on the roots, with $\rho = 0.25$ and $\kappa = 50$ steps. The detector weighs its three signals equally, with $\gamma = 0.1$ and a warm-up of 20 steps, and region selection uses $k_\sigma = 1.5$. The partition is refreshed every 200 steps and installed 20 steps later, with a target region size of 5,000 nodes on `ogbn-arxiv`. The budget N grows with the size of the graph and is 8,192 on `ogbn-arxiv`.

We implement GIST in PyTorch Geometric [43]. Neighbor sampling reuses a cached index that is rebuilt every 20 steps, so its cost per step does not grow with $|E_t|$, and METIS runs through `torch-sparse`. A repair samples its batch and extracts H on the host before the transfer, so that only H moves to the GPU. A test suite checks the update of every supported backbone in float64 against the gradient on the full batch.

IV. Evaluation

We evaluate GIST along the three questions of Section I-A. We first measure the cost of a repair (Q1) and verify that the localized update is identical to the update on the parent graph (Q2). We then measure the effect of repairs on accuracy and isolate the decisions of when to repair and which nodes to retrain in an ablation study (Q3). Finally, we quantify how common evaluation practices inflate the benefit of retraining and characterize the graphs on which GIST saves the most.

A. Experimental Setup

1) Graphs and Streams: We evaluate GIST on the graphs listed in Table II. For node classification, we use six streams. Two of them carry natural drift, namely `ogbn-arxiv` ordered by publication year (`Arxiv-year`), in which papers from 2018–2020 arrive on a graph trained up to 2017, and the `Elliptic` Bitcoin transaction graph, whose 49

TABLE II: Graphs used in the evaluation.

Node classification						Link prediction											
Graph	Drift	$\|V\|$	$\|E\|$	Avg. deg.	Budget N	Graph	Family	$\|V\|$	$\|E\|$	Avg. deg.	Budget N	Graph	Family	$\|V\|$	$\|E\|$	Avg. deg.	Budget N
Arxiv-year	natural	169K	1.17M	13.7	8,192	Facebook	social	4.0K	88K	43.7	100	Gnutella31	p2p	62.6K	148K	4.7	1,564
Elliptic	natural	204K	0.23M	2.3	2,048	Twitter	social	81.3K	1.34M	33.0	2,032	Road-PA	road	1.09M	1.54M	2.8	2,048
Arxiv	injected	169K	1.17M	13.7	8,192	YouTube	social	1.13M	2.99M	5.3	2,048	Road-TX	road	1.38M	1.92M	2.8	2,048
Cora	injected	2.7K	5.3K	3.9	128	Web-Stanford	web	282K	1.99M	14.1	2,048	Road-CA	road	1.97M	2.77M	2.8	2,048
PubMed	injected	19.7K	44K	4.5	128	Web-Google	web	876K	4.32M	9.9	2,048	HepTh	citation	27.8K	352K	25.4	694
Products	injected	2.45M	61.9M	50.5	8,192	Web-BerkStan	web	685K	6.65M	19.4	2,048	HepPh	citation	34.5K	421K	24.4	863
						Gnutella25	p2p	22.7K	54.7K	4.8	567	Patents	citation	3.77M	16.5M	8.8	2,048
						Gnutella30	p2p	36.7K	88.3K	4.8	917						

time steps arrive in order. The other four carry injected drift that covers the three kinds of drift of Section II-B1, namely Cora, PubMed, and ogbn-arxiv (Arxiv) at a scale where whole-graph baselines fit, and ogbn-products (Products) as a scale test. Each node-classification stream runs 1,000 steps, or 2,000 on Cora and PubMed. For link prediction, we use fifteen SNAP graphs from five families, which differ in how far the L-hop neighborhood of a set of nodes reaches. Each of these streams runs 1,000 steps, in which the citation graphs arrive in time order and the others carry injected drift.

2) Backbones and Baselines: Every stream runs with four unmodified two-layer backbones behind the interface of Section III-A, namely GAT, GraphSAGE, GCN, and GIN. GAT does not fit on Products, where one warm-up step on its sampled batch needs 78 GB, so the Products results use three backbones. We compare GIST with FROZEN, the model with fixed parameters whose predictions an exact incremental inference system would serve, and with FULL, the standard sampled update on N uniform roots with fan-out $[25, 15]$, triggered by the same detector as GIST. FULL runs inside the runtime loop of Algorithm 1 and differs from GIST only in the roots that it draws. We further compare thirteen prior update rules, re-implemented inside the same runtime so that they share the stream, the trigger, and the budget and differ only in how an update is computed. These are whole-graph retraining as in InstantGNN and D3-GNN, the live update of ROLAND, the continual learning methods ER-GNN, EWC, and TWP, and the scalable training methods Online CGL, Cluster-GCN, GraphSAINT, VR-GCN, GNNAutoScale, IGLU, and LMC [44]. Every method places its loss on the roots only, so all of them optimize the same objective.

3) System and Protocol: All experiments run on one server with two NVIDIA L40S GPUs (48 GB each), two AMD EPYC 9754 CPUs, and 503 GB of RAM, with PyTorch 2.5.1 and PyTorch Geometric 2.8. Every job is pinned to one GPU and to 8 CPU threads. The GPUs are shared with other users, so we report node, FLOP, and memory counts for every run, since sharing does not affect them, and take wall-clock speedups only from runs that had a GPU to themselves. Unless stated otherwise, GIST takes its parent batch from the sampler and fan-outs of FULL, which makes the two directly comparable, whereas the exactness audits use the whole graph as parent. Every experiment is repeated on ten seeds that were used for nothing else, and the hyperparameters of every method were fixed beforehand on three separate seeds. Differences are paired by seed on one replayed stream and reported as mean $\pm$ 95% confidence interval. Accuracy is measured on holdout nodes that are never trained on, and on Elliptic, where only about 10% of the labeled nodes belong to the illicit class, we also report macro-F1.

TABLE III: Cost of one repair under FULL and GIST (median over 10 seeds).

Stream	Model	Nodes (K)	GFLOPs	Peak MB	Time (ms)	Speedup
Arxiv-yr	GAT	80.4→**55.8**	175→**120**	14,641→**8,587**	353→**236**	1.50
	SAGE	81.0→**57.2**	42.6→**30.0**	726→**473**	236→**158**	1.50
	GCN	80.5→**56.7**	21.6→**15.2**	1,110→**703**	233→**163**	1.43
	GIN	80.9→**57.8**	85.3→**60.9**	806→**547**	237→**169**	1.41
Arxiv	GAT	91.8→**58.2**	197→**124**	13,006→**7,272**	303→**191**	1.59
	SAGE	91.8→**59.8**	48.1→**31.2**	701→**435**	189→**113**	1.67
	GCN	91.8→**58.5**	24.4→**15.6**	1,027→**606**	186→**123**	1.51
	GIN	91.8→**58.0**	96.6→**61.0**	792→**500**	186→**110**	1.69
Elliptic	GAT	17.8→**11.0**	37.1→**22.9**	1,471→**956**	36.1→**26.8**	1.35
	SAGE	17.8→**10.3**	9.2→**5.3**	118→**77**	19.4→**15.1**	1.29
	GCN	17.7→**10.9**	4.6→**2.9**	147→**100**	19.5→**17.8**	1.10
	GIN	17.8→**10.4**	18.6→**10.9**	166→**106**	19.2→**16.3**	1.18
Products	SAGE	748→**471**	341→**215**	4,503→**3,002**	1,953→**1,483**	1.32
	GCN	747→**470**	173→**110**	6,350→**4,282**	2,005→**1,505**	1.33
	GIN	747→**473**	760→**481**	5,230→**3,498**	1,976→**1,549**	1.28

B. Repair Cost

We first measure the cost of one repair at the same budget N (Table III). On the two Arxiv streams, a GIST repair materializes 64–71% of the nodes of a FULL repair and needs 57–68% of its memory, and its FLOPs follow its nodes to within one percentage point. On Arxiv-year, for instance, a GAT repair touches 55.8K instead of 80.4K nodes and peaks at 8.6 instead of 14.6 GB. The ratio is 58–62% on Elliptic, 63% on Products, 51–53% on Cora, and 70–76% on PubMed, where a 128-root update is small either way. The repair times in Table III were recorded inside the streams on shared GPUs, so we analyze speed below on runs with a dedicated GPU.

The saving follows from the observation of Section II-B3 that the cost of an update is set by the L-hop neighborhood of its roots, and this neighborhood grows quickly with the number of uniform roots (Fig. 6(a)). The 2-hop neighborhood of uniform roots covers 35% of ogbn-arxiv at $N = 128$, 84% at $N = 8{,}192$, and 96% at $N = 32{,}768$, and 82% of ogbn-products at $N = 8{,}192$, so at practical budgets a uniform update touches most of the graph. The same number of roots drawn from one region covers 61% and 25% of the two graphs at $N = 8{,}192$. Across six graphs at

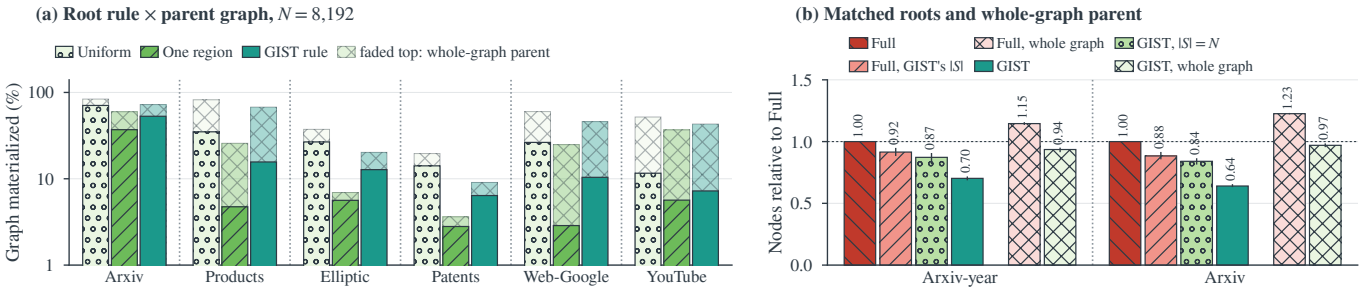


Fig. 5: Cost of one update. (a) Share of the graph materialized by 8,192 roots drawn uniformly, from one region, or by GIST's rule, with a fan-out-limited batch as parent and with the whole graph as parent (faded top). (b) Nodes per repair relative to FULL, mean over backbones. GAT on `Arxiv-year` with the whole-graph parent runs out of memory and is omitted.

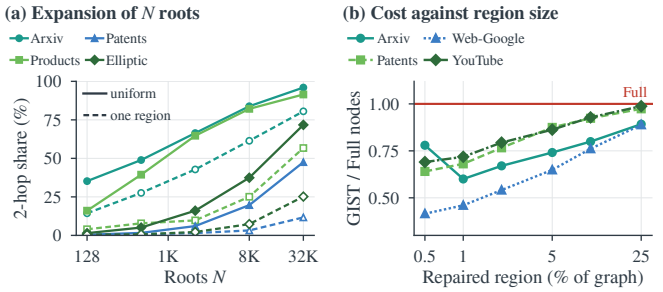


Fig. 6: The cost of an update grows with the L-hop expansion of its roots. (a) Share of the graph in the exact 2-hop neighborhood of N uniform or regional roots. (b) GIST's update against the size of the repaired region (GAT, $N = 2{,}048$).

$N = 8{,}192$, uniform roots materialize 71% of `ogbn-arxiv` with a fan-out-limited parent and 84% with the whole graph as parent, whereas roots from one region materialize 37% and 60% (Fig. 5(a)). GIST's rule, which adds a rehearsal share of uniform roots to the regional ones, lies in between, at 0.39–0.75× the nodes of uniform roots with a fan-out parent and 0.46–0.86× with the whole graph as parent.

Since the budget N is an upper bound on the number of roots, GIST uses a median of 59–62% of it, because the drifted regions hold fewer training nodes than their share. We therefore separate the saving into fewer roots and shared neighborhoods (Fig. 5(b)). FULL on as many uniform roots as GIST drew costs 0.86–0.94× a FULL repair, and GIST costs 0.71–0.78× of that, so most of the saving comes from roots that share their neighborhoods. Topped up to exactly N roots, GIST still costs 0.82–0.90× FULL. The same holds for the parent graph. On the two `Arxiv` streams, GIST computes the exact whole-graph gradient on its roots at 0.92–0.98× the nodes of a sampled FULL repair, whereas FULL with the whole graph as parent costs 1.14–1.23×. On the denser `Products`, `Web-Google`, and `YouTube`, the whole-graph parent costs 1.7–3.7× a sampled update, but GIST's rule still materializes only 0.77–0.83× the nodes of uniform roots with the same parent.

The cost of a repair also follows the size of the region that is repaired (Fig. 6(b)). On four graphs, a GAT update at $N = 2{,}048$ grows from 0.42–0.78× FULL's nodes for a region of 0.5% of the graph to 0.89–0.99× for a region of 25%, while FULL's update stays the same size. Whole-graph retraining pays for the graph instead, with 2.4–25× FULL's nodes on these graphs, and its GAT update runs out of memory on all four.

Fewer nodes translate into shorter repairs when fixed costs do not dominate. With a dedicated GPU, a GIST repair at $N = 8{,}192$ is 1.59–1.77× faster than a FULL repair, and no single seed falls below 1.40× (Fig. 7(a)). The speedup grows with the budget, from 1.26–1.66× at $N = 2{,}048$, while the node ratio stays near 63%. Region selection adds 10–13 ms per repair, and including it, the speedup is 1.46–1.61× at $N = 8{,}192$. Neighbor sampling on the host takes 167–171 ms of a FULL repair and 88–98 ms of a GIST repair (Fig. 7(b)). For GraphSAGE, GCN, and GIN, sampling accounts for 80–91% of the repair and the remainder does not shrink, so their speedup is a saving in host-side sampling with the CPU sampler of PyG. GAT, whose model step is large, saves on both parts, with sampling falling from 171 to 98 ms and the remainder from 125 to 85 ms. Whole-graph retraining skips sampling and is faster per repair while it fits (35–44 ms for GraphSAGE, GCN, and GIN), but at 6.3–9.7× FULL's nodes.

The smaller update also determines whether a repair fits in GPU memory at all (Fig. 7(c)). For GAT at $N = 8{,}192$ on `Arxiv`, FULL runs out of memory at hidden width 1,024 and the whole-graph update already at 512, whereas GIST completes at every width with at most 36.4 GB. For GraphSAGE on `Products`, where both sampled updates fit at every width, GIST needs 0.47–0.48× FULL's memory (7.3 against 15.5 GB at width 1,024), and the whole-graph update never fits.

A repair still competes with serving while it runs (Fig. 7(d)). With a repair running on the serving GPU, the 99th-percentile latency rises 2.4× for GAT and 5.3× for GraphSAGE under every rule, and GIST ends each repair 1.59–1.77× sooner. Monitoring reuses the forward pass of serving and costs

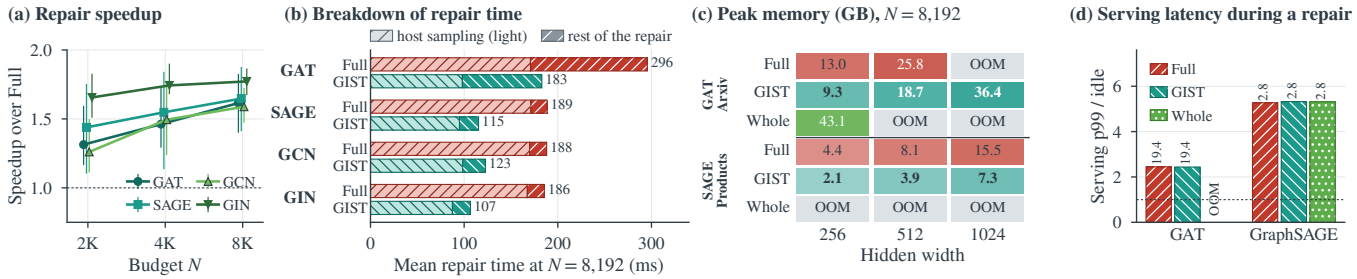


Fig. 7: Speed and memory of a repair on `Arxiv`, with (a), (b), and (d) on seeds 0–4, each run with a GPU to itself. (a) Repair speedup over FULL, mean of per-seed ratios with min–max. (b) Mean repair at $N = 8{,}192$, split into neighbor sampling on the host and the rest. (c) Peak repair memory in GB of GAT on `Arxiv` and GraphSAGE on `Products`, or out of memory (OOM) on every seed. (d) Serving p99 while a repair runs on the serving GPU.

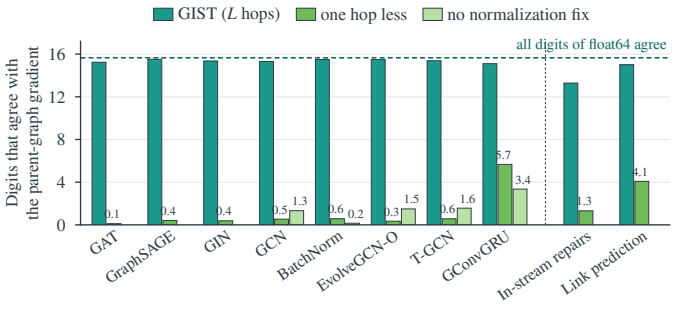


Fig. 8: Digits of the parent-graph gradient that an update reproduces in float64, for GIST (worst audit) and for two controls that each break one exactness condition (best audit).

0.35–0.37 ms per step, about 1% of a stream step. This is 4.8% of GAT's forward pass but 37–44% of the forward passes of GraphSAGE, GCN, and GIN, which take less than 1 ms. End to end, GIST is 0.6–1.5 ms per step faster than FULL, which lies within the run-to-run variation of 1.5–2.9 ms.

C. Update Exactness

We next test the property on which every saving rests. For each backbone, we compare the gradient of the loss on the roots, computed on the L-hop subgraph of GIST, with the gradient on the parent graph in float64, and count how many decimal digits each update reproduces, out of the 15.7 digits given by the machine epsilon of float64 (Fig. 8). Two controls each break one condition of Section II-B4, by cutting the subgraph to $L - 1$ hops or by letting the layers normalize over the subgraph itself. On all eight backbones, the localized gradient reproduces 15.1–15.5 digits, that is, it differs from the parent gradient by at most 7.9×10^{-16}, which is within the round-off of float64. One hop less reproduces at most 0.6 digits on every backbone except GConvGRU. Without the corrections of Section III-C, GCN, BatchNorm, EvolveGCN-O, and T-GCN reproduce only 0.2–1.6 digits, and with them they are exact. GConvGRU shows why the depth must count every convolution inside a recurrent cell, since its subgraph must span four hops, and three hops reproduce only 5.7 digits. The property also holds inside the streams. All 69 audited repairs of the node-classification streams reproduce at least 13.3 digits, a difference of at most 5.2×10^{-14}, and all 180 audited updates of the link-prediction graphs at least 15.0, while their controls reproduce at most 1.3 and 4.1 digits. Three-layer GAT and GraphSAGE models are exact at three hops ($\leq 9.2 \times 10^{-16}$).

The scalable training methods of Section II-A also bound the cost of a step, but each of them computes a different gradient (Fig. 2). On the label-only streams, every rule computes the same repair, with the same roots and the same warm model, for 20 steps. The median gradient error is 4.3% for neighbor sampling at the fan-out of FULL, 17.3% for Online CGL, 7.9–9.2% for Cluster-GCN, GraphSAINT, and VR-GCN, and 32.5% for GNNAutoScale. After 20 steps, the resulting models disagree with the exact repair on 4.1–10.3% of the holdout predictions, and the disagreement grows with staleness, from 9.1 to 16.0% for GNNAutoScale when its histories are refreshed every 10 steps instead of every step, from 9.6 to 29.5% for IGLU at five steps of staleness instead of one, and from 10.8 to 19.7% for LMC. Their accuracy is close to that of the exact repair on the drifted nodes (38.0–43.6% against 42.5%) and mostly lower on the clean ones (45.1–60.0% against 59.7%). The two rules that come closest, Cluster-GCN and GNNAutoScale on Cluster-GCN batches, materialize 4.0× the nodes of the exact subgraph. In the same experiment, GIST matches the parent gradient to within 7.9×10^{-15} in all 120 audited repairs.

D. Accuracy under Drift

We now measure the effect of repairs on holdout accuracy over a whole stream, which depends on the kind of drift (Fig. 9(a)). On natural drift, retraining helps. On `Arxiv-year`, FULL gains $+0.59 \pm 0.17$ pp over the frozen model, significantly for all four backbones and up to $+1.03$ pp for GAT, and EWC, TWP, and ROLAND gain $+0.45$ to $+0.71$ pp (Fig. 9(b)). On `Elliptic`, accuracy hides the gain because the illicit class is small, but macro-F1 rises by $+2.27 \pm 0.71$ pp under FULL and by $+1.81 \pm 0.67$ pp under

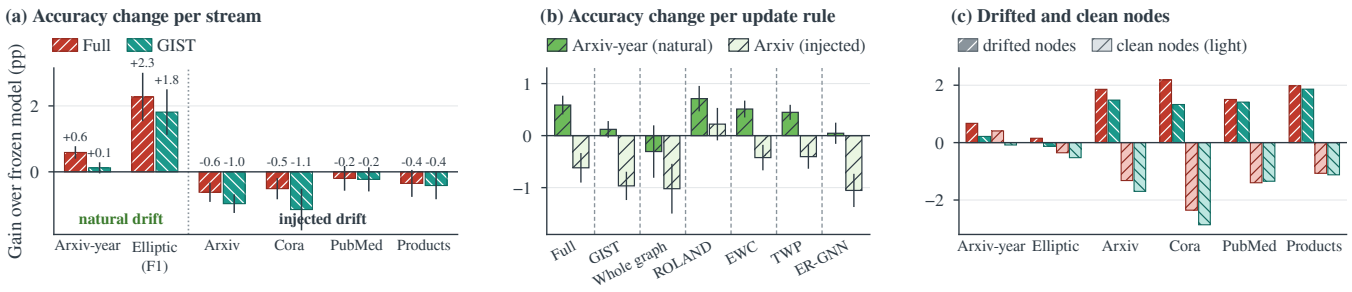


Fig. 9: Holdout accuracy against the frozen model over a whole stream (pp, mean ± 95% CI over backbones × seeds). (a) FULL and GIST on every stream, `Elliptic` in macro-F1. (b) Every update rule on the two `Arxiv` streams. (c) Nodes drifted in the last 200 steps and clean nodes.

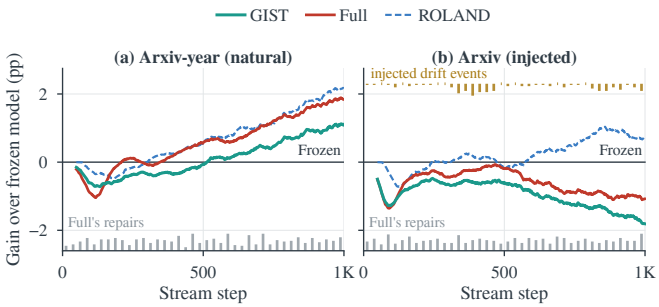


Fig. 10: Holdout accuracy along the stream against the frozen model (50-step mean over backbones × seeds). Bars show how often FULL repairs and, on `Arxiv`, when drift is injected, per 25 steps.

GIST, and by up to +4.1 pp for GAT under FULL, whose recall on the illicit class rises from 48% to 63%. On injected drift, no rule significantly improves on the frozen model. FULL loses 0.20–0.62 pp on the four streams, whole-graph retraining and ER-GNN lose more on `Arxiv`, and only ROLAND ends above the frozen model there, without significance ($+0.22 \pm 0.30$ pp).

The split into drifted and clean nodes explains this difference (Fig. 9(c)). On the injected streams, a repair recovers the drifted nodes, by +1.5 to +2.2 pp under FULL, but since all nodes share the parameters, it also moves the clean nodes, which lose 1.1–2.4 pp and form the majority. On `Arxiv-year`, where the whole graph drifts slowly, both groups gain.

The effect builds up along the stream (Fig. 10). On `Arxiv-year`, FULL and GIST start below the frozen model, pass it after about 300 and 500 steps, and end 1.8 and 1.1 pp above it. On `Arxiv`, both fall further behind as the injected events accumulate and end 1.1 and 1.8 pp below it. ROLAND ends above the frozen model on both streams, but its whole-stream gain on `Arxiv` is not significant.

Throughout, GIST follows FULL closely. With two one-sided tests at a margin of 2 pp, GIST is equivalent to FULL in 22 of the 23 backbone–stream cells, and the exception is GIN on `Cora` (-2.14 ± 1.22 pp). Since this margin exceeds the gain of FULL on `Arxiv-year`, we also report the measured gap directly. GIST remains slightly below FULL, by 0.47 ± 0.09 pp on `Arxiv-year`, 0.35 ± 0.20 pp on `Arxiv`, 0.63 ± 0.43 pp on `Cora`, and less than 0.1 pp on `PubMed` and `Products`, and its macro-F1 on `Elliptic` is 0.31–0.62 pp lower. Since the gradient is exact, this gap comes from which roots are retrained rather than from how the update is computed, which we isolate in Section IV-E.

E. Ablation Study

We now remove or replace each design choice of GIST in turn on the two `Arxiv` streams. The first choice is which nodes to retrain (Fig. 11(a)). FULL on as many uniform roots as GIST drew is as accurate as FULL (-0.02 and -0.01 pp) at 0.86–0.94× its nodes, so the smaller number of roots does not explain the gap of GIST. Topping GIST up to N roots narrows the gap to 0.23–0.36 pp at 0.84–0.87× the nodes. Drawing the roots from random regions instead of the drifted ones costs the same and is as accurate as GIST (-0.42 against -0.47 pp on `Arxiv-year`). The saving and the gap therefore both come from retraining coherent regions rather than from locating the region that drifted.

The rehearsal share ρ trades between the two (Fig. 11(b)). Without rehearsal, an update costs 0.28–0.39× FULL's nodes and trails it by 1.3–1.4 pp. At the default $\rho = 0.25$, it costs 0.65–0.67× and trails by 0.42–0.62 pp, and at $\rho = 0.5$, it costs 0.82–0.85× and trails by only 0.22–0.32 pp. Without rehearsal, the cost ratio falls as the budget grows, since the overlap within one region grows faster than the neighborhood of uniform roots. When each drift event is scattered over random nodes instead of one region, the cost ratio barely changes (0.63–0.64×), so the saving requires a coherent set of roots rather than localized drift.

The second choice is when to repair (Fig. 11(c)). The cooldown κ and the threshold τ move the number of repairs on `Arxiv-year` from 7 to 22 per 1,000 steps. Both FULL and GIST gain more with more repairs, FULL from +0.32 to +0.82 pp and GIST from -0.15 to +0.32 pp, while the gap between them stays at 0.43–0.52 pp. GIST on a timer at

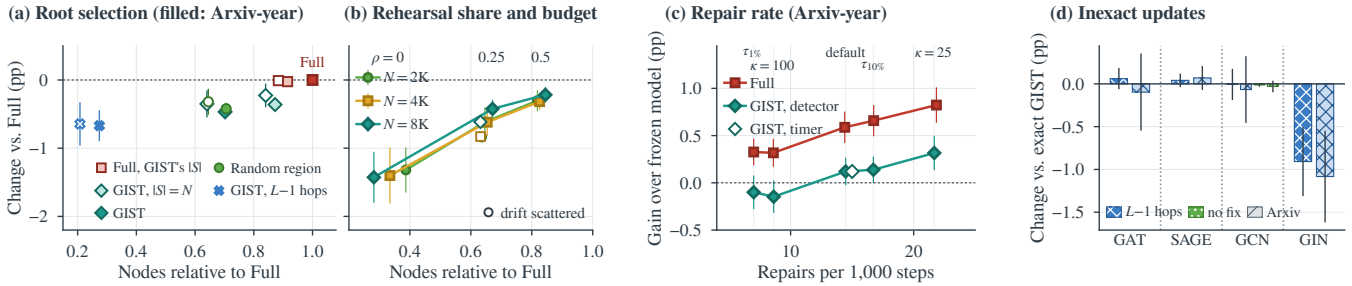


Fig. 11: Ablation study (10 seeds, mean ± 95% CI). (a) Root rule, mean over the four backbones. (b) Rehearsal share ρ and budget N, mean over GAT and GraphSAGE on both `Arxiv` streams, with hollow markers for drift scattered over random nodes. (c) Repair rate, set by the cooldown κ and the threshold τ, mean over the four backbones, with hollow markers for GIST on a timer. (d) Accuracy of an update that breaks one exactness condition, against exact GIST.

TABLE IV: Accuracy gain of FULL over the frozen model with and without each evaluation practice.

Practice	Stream	With	Without
Under-trained start	`Cora`	+9.32±1.53	−0.56±0.32
	`Arxiv-year`	+15.26±2.78	+0.83±0.15
Shared learning rate	`Cora`	+5.23±1.55	−0.56±0.72
128-root budget (coverage)	`Arxiv`	2.2%	100%
Halo supervision	`Arxiv`	−1.53±0.31	−0.39±0.10
Asymmetric warm-up	`Arxiv-year`	+1.44±0.44	+0.59±0.17

the rate of FULL is as accurate as GIST with the detector (+0.12 pp for both), and a trigger that fires exactly on the injected drift events of `Arxiv` makes half as many repairs and still does not improve on the frozen model. The detector flags a step in 92–97% of all 50-step windows, so the cooldown sets the rate of repairs. Accuracy therefore depends on how often the model is repaired, and the trigger is a policy that can be replaced without changing the update (Section III-B1).

The third choice is to keep the update exact (Fig. 11(d)). An update on the $(L-1)$-hop subgraph of the same roots materializes only 0.32–0.42× the nodes of the update of GIST. For GAT, GraphSAGE, and GCN, its accuracy is within 0.1 pp of exact GIST (−0.10 to +0.07 pp), whereas GIN loses 0.91–1.08 pp. GCN without the normalization fix is as accurate as with it (−0.01 and −0.03 pp). On these streams, breaking exactness therefore changes the update but rarely its accuracy. The parent graph matters as well. With the whole graph as parent, the repairs of GraphSAGE and GAT are as accurate as with a sampled parent, those of GCN gain 0.2–0.5 pp, and those of GIN lose 2.3–3.4 pp against FULL under both rules, so the choice of parent depends on the backbone rather than on GIST.

F. Impact of Evaluation Practices

The finding that retraining rarely helps on injected drift in node classification contradicts much of the literature, so we measure the practices that produce the opposite finding (Table IV), in percentage points over four backbones and ten seeds, and with GAT and GraphSAGE for the under-trained

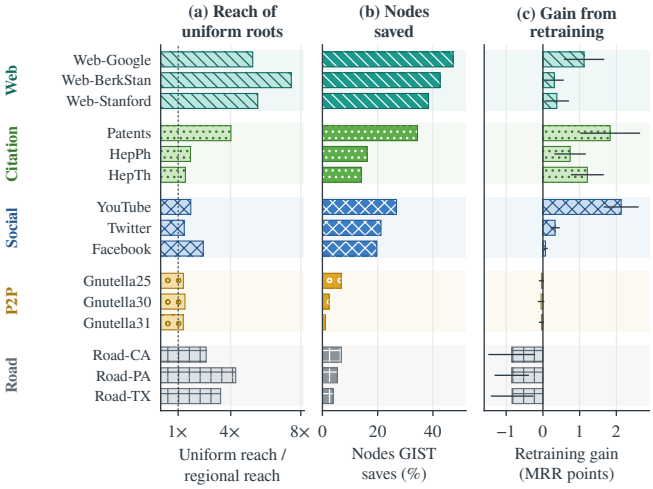


Fig. 12: Link prediction on fifteen graphs (4 backbones × 10 seeds). (a) Coverage of the 2-hop neighborhood of uniform roots relative to that of regional roots. (b) Nodes that GIST saves against FULL. (c) Gain of FULL over the frozen model in MRR points.

start on `Arxiv-year`. Starting the stream from a model warmed up for 5 epochs instead of until convergence turns the change of FULL on `Cora` from −0.56 into +9.32 pp, and on `Arxiv-year` from +0.83 into +15.26 pp, because the repairs then continue the training rather than adapt the model. Using the adaptation learning rate for the warm-up as well lowers the frozen baseline and turns the change on `Cora` into +5.23 pp. A 128-root update, common in prior evaluations, covers only 2.2% of the drifted training nodes of an `Arxiv` event, so its effect cannot be distinguished from noise. Placing the loss on the sampled halo as well as on the roots trains 86% of its targets on truncated neighborhoods and costs 1.1 pp on structural drift. Warming up on one edge direction and streaming on both inflates the gain of FULL on `Arxiv-year` from +0.59 to +1.44 pp. With all five practices removed, retraining does not significantly improve on the frozen model

in any of the 15 injected-drift cells of node classification, and improves significantly in 5 of its 8 natural-drift cells.

G. Impact of Graph Structure

The saving of GIST depends on how much further uniform roots reach than roots from one region, which is a property of the graph (Fig. 12(a)–(b)). Where the 2-hop neighborhood of uniform roots covers 4–$7.5\times$ as much of the graph, on the web graphs and `Patents`, GIST saves 34–47% of the nodes of FULL. At 1.4–$2.4\times$, on the social graphs and the two smaller citation graphs, it saves 14–27%, and on the peer-to-peer graphs, at 1.3–$1.4\times$, it saves only 1–7%. Road networks are the exception. Their regions are compact, yet GIST saves only 4–7%, because the drifted regions supply only 33–58% of the training edges of an update. Whether retraining pays follows the same families (Fig. 12(c)). FULL gains significantly on the social, web, and citation graphs, up to $+2.13$ MRR points on `YouTube`, is neutral on the peer-to-peer graphs, and hurts on the road networks (-0.85). Where retraining pays, GIST trails FULL by 0.02–0.75 MRR points.

V. DISCUSSION

A. Implications

GIST separates a guarantee from a policy. The localized update computes the same gradient as the graph from which its subgraph is extracted, which our audits confirm to within 10^{-13} on every backbone (Section IV-C). The trigger and the selection of roots, in contrast, are policies that can be replaced without affecting this guarantee, and GIST on a timer is as accurate as GIST with its detector (Section IV-E). Within this separation, the cost of a repair depends on which nodes are retrained rather than how many. At $N = 8{,}192$, the 2-hop neighborhood of uniform roots covers 84% of `ogbn-arxiv`, whereas the same number of roots from one region covers 61% (Section IV-B), and roots from a random region cost the same and are as accurate as roots from the drifted one (Section IV-E). The choice of roots therefore deserves the same attention as the choice of when to retrain.

Exactness matters because the effect of an approximation depends on the model. Every cheaper method that we evaluated disagrees with the exact repair on 4.1–10.3% of the holdout predictions after 20 steps (Section IV-C), and dropping one hop of context changes accuracy by less than 0.1 pp for GAT, GraphSAGE, and GCN but costs GIN 0.91–1.08 pp (Section IV-E). The exact update removes this dependence, and with the whole graph as the source of the subgraph, it costs 0.92–$0.98\times$ the nodes of a sampled update on the `Arxiv` streams (Section IV-B).

Neither a cheaper nor an exact repair helps, however, when retraining itself does not pay. Retraining improves accuracy under natural drift, by $+0.59$ pp on `Arxiv-year` and by $+2.27$ pp in macro-F1 on `Elliptic`, whereas under injected label drift in node classification, a repair recovers the drifted nodes by 1.5–2.2 pp but costs the clean majority 1.1–2.4 pp (Section IV-D). The graphs on which retraining pays are also those on which GIST saves the most, with 34–47% of the nodes on web graphs and `Patents` against 1–7% on peer-to-peer graphs (Section IV-G). Evaluation practice can also reverse these conclusions, since an under-trained starting model alone turns the change on `Cora` from -0.56 into $+9.32$ pp (Section IV-F).

B. Limitations

GIST has five limitations. Its detector reads only the outputs of the served model, so drift that changes labels without changing inputs must be signaled by another source, such as the arrival of new labels on training nodes. Since GIST retrains different roots than uniform sampling, its accuracy remains below that of sampled retraining by less than 0.7 pp, and a rehearsal share of 0.5 narrows this gap to 0.22–0.32 pp at 0.82–$0.85\times$ the cost of sampled retraining (Section IV-E). Memory-based temporal GNNs have no finite depth and therefore lie outside the interface of GIST (Section III-C). Exact repairs on the whole graph remain bounded by GPU memory. Finally, a repair raises the 99th-percentile serving latency by 2.4–$5.3\times$ while it runs, which GIST shortens but does not lower, and on light backbones 80–91% of a repair is host-side sampling (Section IV-B).

VI. CONCLUSION

In this paper, We study how to repair the parameters of a deployed GNN under drift while it continues to serve. We present GIST, a runtime that computes each repair of an unmodified model on the subgraph induced by the L-hop neighborhood of the retrained nodes. Since the output of an L-layer GNN at a node depends only on its L-hop neighborhood, this gradient equals the gradient on the graph from which the subgraph is extracted, verified to within 10^{-13}, and the cost of a repair follows the neighborhood of its roots rather than the graph size. On the two `ogbn-arxiv` streams, at the budget of sampled retraining, GIST materializes 64–71% of the nodes and 57–68% of the memory and runs 1.59–$1.77\times$ faster. When the subgraph is extracted from the whole graph, GIST computes the exact whole-graph gradient at about the cost of a sampled update. Its accuracy is statistically equivalent to sampled retraining in 22 of 23 settings, and its savings are largest on social, web, and citation graphs, where retraining also improves accuracy. In future work, we plan to reduce host-side sampling, to schedule repairs around the serving path, to draw rehearsal roots from already materialized neighborhoods, and to predict from a graph and its stream whether regional repair will save work.

Note on AI Usage: Grammarly was used for grammar and stylistic refinement only, and ChatGPT was used for refining the quality of figures and plots. All scientific content was written and verified by the authors.

REFERENCES

[1] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[2] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[3] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018.

[4] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *International Conference on Learning Representations (ICLR)*, 2019.

[5] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2018, pp. 974–983.

[6] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics," KDD Workshop on Anomaly Detection in Finance, 2019.

[7] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open graph benchmark: Datasets for machine learning on graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 22 118–22 133.

[8] M. Besta, M. Fischer, V. Kalavri, M. Kapralov, and T. Hoefler, "Practice of streaming processing of dynamic graphs: Concepts, models, and systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 6, pp. 1860–1876, 2023.

[9] D. Wu, Z. Li, and T. Mitra, "Inkstream: Instantaneous gnn inference on dynamic graphs via incremental update," in *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2025, pp. 1273–1285.

[10] P. Naman and Y. Simmhan, "Ripple: Scalable incremental gnn inferencing on large streaming graphs," in *2025 IEEE 45th ICDCS*. IEEE, 2025, pp. 857–867.

[11] L. Zhang, P. Qiao, Z. Zhang, Y. Yuan, and G. Wang, "StreamTGN: A GPU-efficient serving system for streaming temporal graph neural networks," 2026.

[12] Y. Zheng, H. Wang, Z. Wei, J. Liu, and S. Wang, "Instant graph neural networks for dynamic graphs," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 2605–2615. [Online]. Available: https://doi.org/10.1145/3534678.3539352

[13] R. Guliyev, A. Haldar, and H. Ferhatosmanoglu, "D3-GNN: Dynamic distributed dataflow for streaming graph neural networks," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, pp. 2764–2777, 2024.

[14] J. You, T. Du, and J. Leskovec, "ROLAND: Graph learning framework for dynamic graphs," in *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2022, pp. 2358–2366.

[15] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[16] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019.

[17] M. Wu, X. Zheng, Q. Zhang, X. Shen, X. Luo, X. Zhu, and S. Pan, "Graph learning under distribution shifts: A comprehensive survey on domain adaptation, out-of-distribution, and continual learning," 2024.

[18] S. M. Kazemi, R. Goel, K. Jain, I. Kobyzev, A. Sethi, P. Forsyth, and P. Poupart, "Representation learning for dynamic graphs: A survey," *Journal of Machine Learning Research*, vol. 21, no. 70, pp. 1–73, 2020.

[19] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh, "Cluster-GCN: An efficient algorithm for training deep and large graph convolutional networks," in *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019, pp. 257–266.

[20] G. Donghi, L. Pasa, D. Zambon, C. Alippi, and N. Navarin, "Online continual graph learning," 2025.

[21] F. Zhou and C. Cao, "Overcoming catastrophic forgetting in graph neural networks with experience replay," in *AAAI Conference on Artificial Intelligence*, vol. 35, no. 5, 2021, pp. 4714–4722.

[22] H. Liu, Y. Yang, and X. Wang, "Overcoming catastrophic forgetting in graph neural networks," in *AAAI Conference on Artificial Intelligence*, vol. 35, no. 10, 2021, pp. 8653–8661.

[23] Y. Liu, R. Qiu, and Z. Huang, "Cat: Balanced continual graph learning with graph condensation," in *IEEE International Conference on Data Mining (ICDM)*. IEEE, 2023, pp. 1157–1162.

[24] X. Zhang, D. Song, and D. Tao, "CGLB: Benchmark tasks for continual graph learning," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

[25] G. Sun, X. Zhang, J. Ni, and D. Song, "DRIFT: A benchmark for task-free continual graph learning with continuous distribution shifts," 2026.

[26] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. Prasanna, "GraphSAINT: Graph sampling based inductive learning method," in *International Conference on Learning Representations (ICLR)*, 2020.

[27] M. Fey, J. E. Lenssen, F. Weichert, and J. Leskovec, "GNNAutoScale: Scalable and expressive graph neural networks via historical embeddings," in *International Conference on Machine Learning (ICML)*, 2021, pp. 3294–3304.

[28] S. D. Narayanan, A. Sinha, P. Jain, P. Kar, and S. Sellamanickam, "IGLU: Efficient GCN training via lazy updates," in *International Conference on Learning Representations (ICLR)*, 2022.

[29] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. B. Schardl, and C. E. Leiserson, "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *AAAI Conference on Artificial Intelligence*, vol. 34, no. 4, 2020, pp. 5363–5370.

[30] Y. Seo, M. Defferrard, P. Vandergheynst, and X. Bresson, "Structured sequence modeling with graph convolutional recurrent networks," in *International Conference on Neural Information Processing (ICONIP)*, 2018, pp. 362–373.

[31] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, and H. Li, "T-GCN: A temporal graph convolutional network for traffic prediction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 9, pp. 3848–3858, 2020.

[32] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive representation learning on temporal graphs," in *International Conference on Learning Representations (ICLR)*, 2020.

[33] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," 2020.

[34] H. Zhou, D. Zheng, I. Nisa, V. Ioannidis, X. Song, and G. Karypis, "TGL: A general framework for temporal GNN training on billion-scale graphs," *Proceedings of the VLDB Endowment*, vol. 15, no. 8, pp. 1572–1580, 2022.

[35] C. Chen, D. Gao, Y. Zhang, Q. Wang, Z. Fu, X. Zhang, J. Zhu, Y. Gu, and G. Yu, "NeutronStream: A dynamic GNN training framework with sliding window for graph streams," *Proceedings of the VLDB Endowment*, vol. 17, no. 3, pp. 455–468, Nov. 2023. [Online]. Available: https://doi.org/10.14778/3632093.3632108

[36] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[37] J. Chen, J. Zhu, and L. Song, "Stochastic training of graph convolutional networks with variance reduction," in *International Conference on Machine Learning (ICML)*, 2018, pp. 942–950.

[38] M. Zhou, J. Lu, Y. Song, and G. Zhang, "Multi-stream concept drift self-adaptation using graph neural network," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12 828–12 841, 2023.

[39] M. Zhou, J. Lu, P. Lu, and G. Zhang, "Dynamic graph regularization for multi-stream concept drift self-adaptation," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 11, pp. 6016–6028, 2024.

[40] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998.

[41] S. Kumar, X. Zhang, and J. Leskovec, "Predicting dynamic embedding trajectory in temporal interaction networks," in *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2019, pp. 1269–1278.

[42] R. Trivedi, M. Farajtabar, P. Biswal, and H. Zha, "DyRep: Learning representations over dynamic graphs," in *International Conference on Learning Representations (ICLR)*, 2019.

[43] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[44] Z. Shi, X. Liang, and J. Wang, "LMC: Fast training of GNNs via subgraph sampling with provable convergence," in *International Conference on Learning Representations (ICLR)*, 2023.